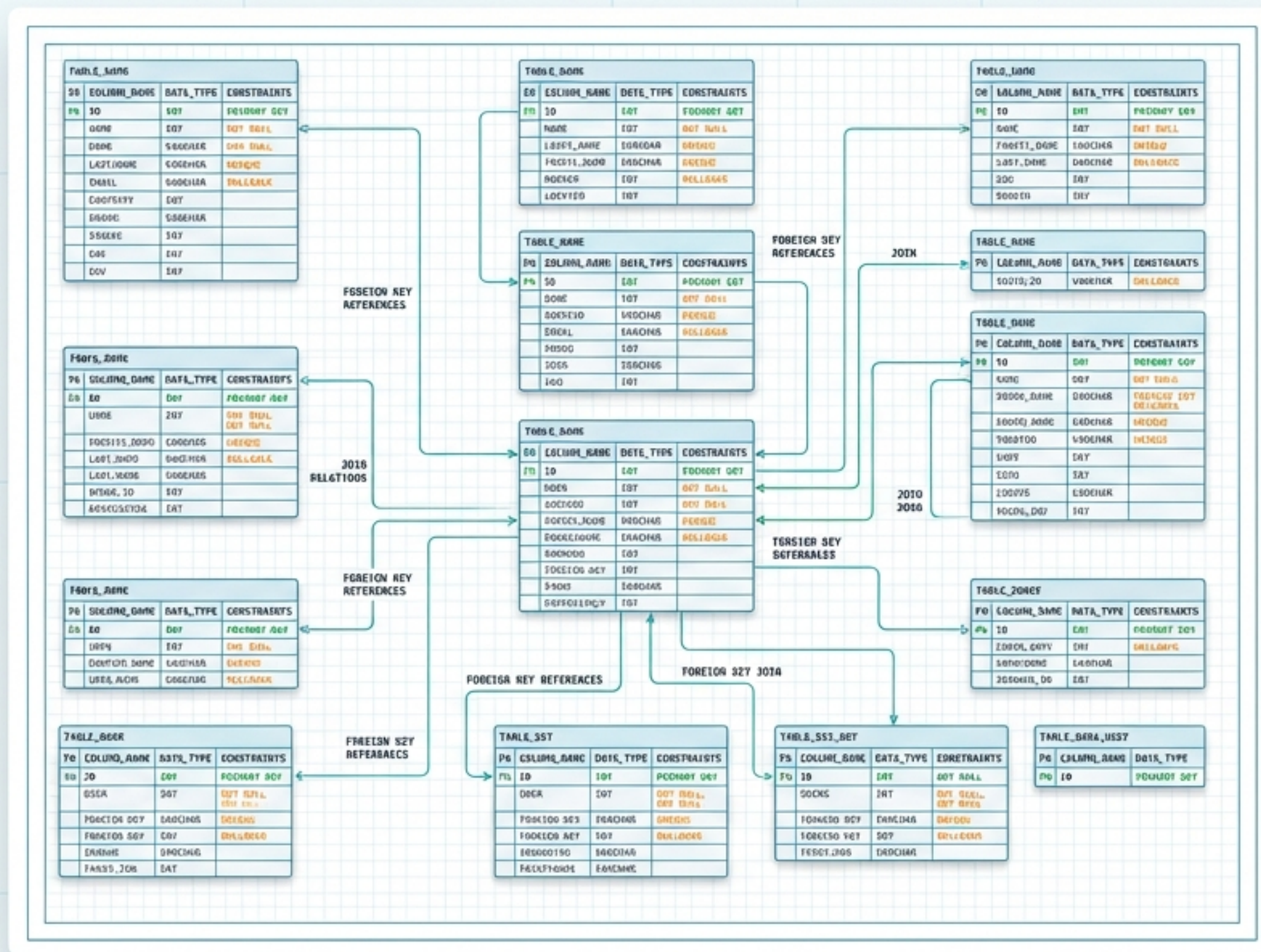


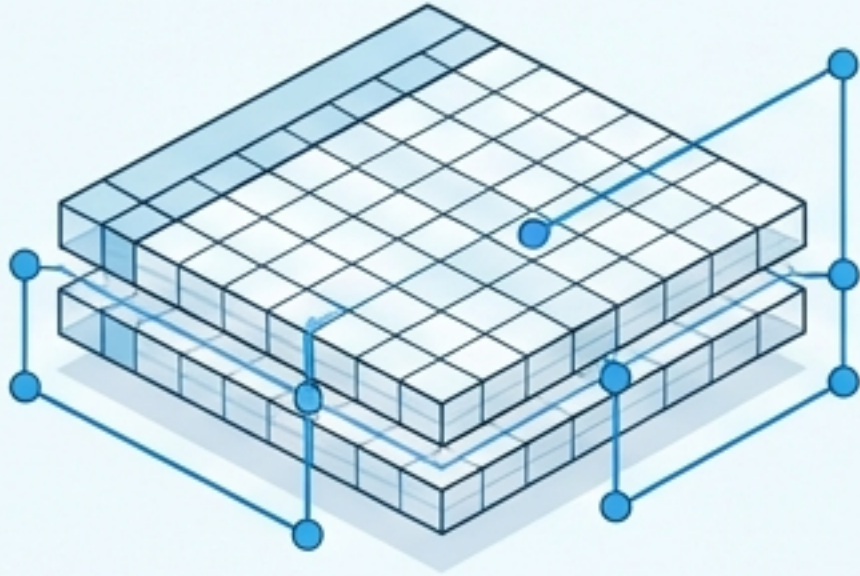
Sổ Tay Cẩm Nang SQL & MySQL

Tối ưu hóa kiến thức nền tảng cho học tập và thực chiến phỏng vấn



Tổng Quan Về Kiến Trúc Dữ Liệu

Cơ Sở Dữ Liệu Quan Hệ (SQL)



Dữ liệu lưu dưới dạng **Bảng (Tables)** gồm **Hàng (Rows)** & **Cột (Columns)**. Các bảng liên kết với nhau qua các **Khóa (Keys)**.

Công nghệ: MySQL, PostgreSQL, SQL Server

Phi Quan Hệ (NoSQL)



Dữ liệu lưu trữ **tự do (JSON, Key-Value, Graph)**. Phù hợp cho cấu trúc dữ liệu linh hoạt, không cố định.

MySQL Server
Động cơ lưu trữ

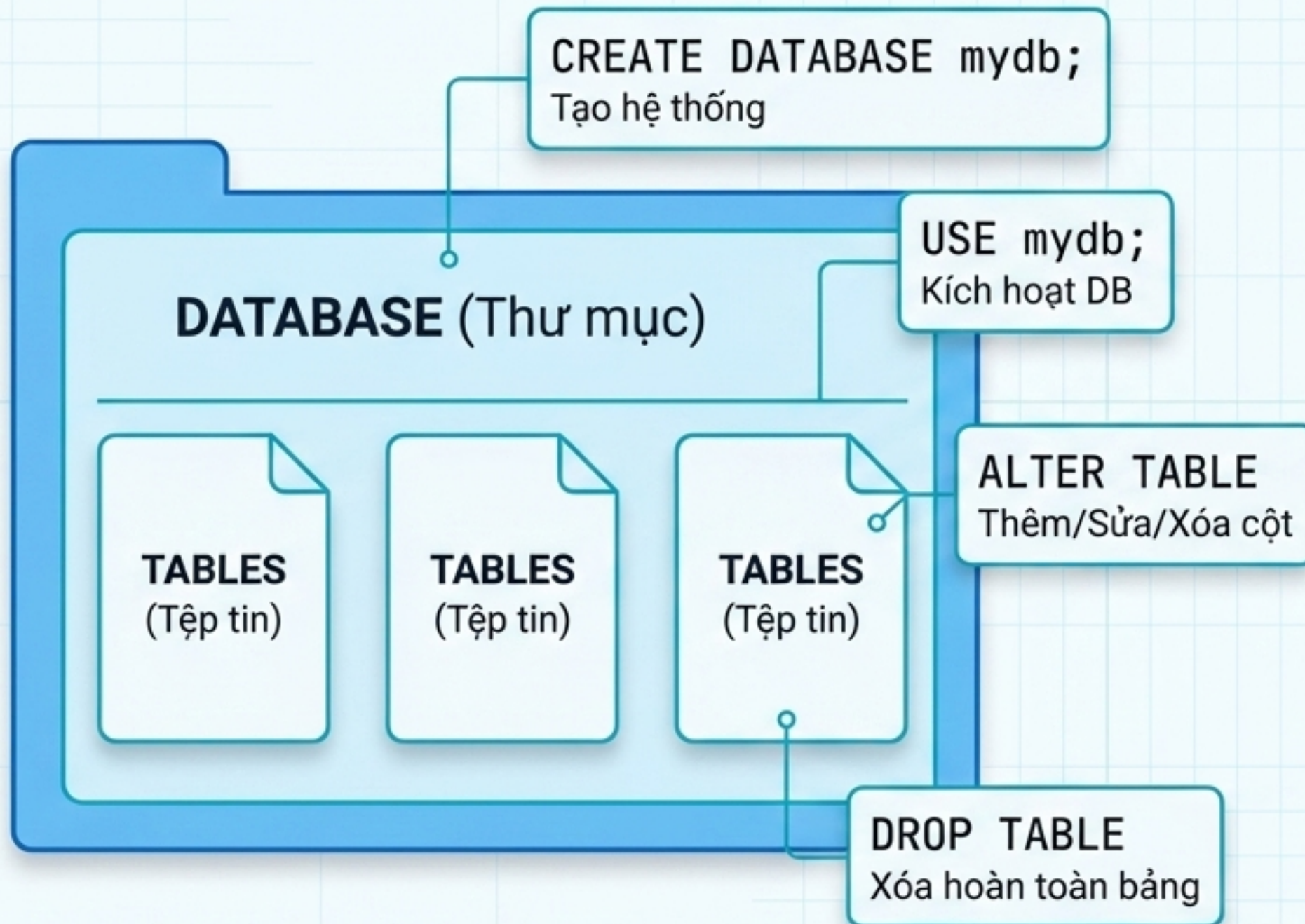


MySQL Workbench
Giao diện quản lý DBMS



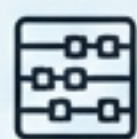
Người Dùng
Client / Ứng dụng

Ngôn Ngữ Định Nghĩa Dữ Liệu (DDL - Xây Dựng Cấu Trúc)



```
CREATE TABLE employees (  
    emp_id INT,  
    first_name VARCHAR(50),  
    hourly_pay DECIMAL(5, 2),  
    hire_date DATE  
);
```


Hệ Thống Kiểu Dữ Liệu Cơ Bản (Data Types)



Số Học (Numeric)

INT

Số nguyên (VD: `emp_id = 1`)

DECIMAL(size, precision)

Số thập phân. Lưu ý:
DECIMAL(5,2) nghĩa là tối đa 5 chữ số, trong đó có 2 chữ số thập phân (VD: 999.99)



Chuỗi Ký Tự (String)

VARCHAR(max_size)

Văn bản có độ dài thay đổi.
Tiết kiệm bộ nhớ hơn so với kích thước cố định.
(VD: VARCHAR(50) cho tên người)



Thời Gian (Temporal)

DATE

Chỉ ngày (YYYY-MM-DD)

TIME

Chỉ giờ (HH:MM:SS)

DATETIME

Kết hợp ngày và giờ. Dùng hàm NOW() để lấy thời gian hiện tại của hệ thống

Ma Trận Ràng Buộc Dữ Liệu (The Constraints Matrix)

Ràng Buộc (Constraint)	Định Nghĩa Lỗi	Chấp Nhận Null?	Chấp Nhận Trùng?
PRIMARY KEY	Định danh duy nhất cho mỗi hàng (Chỉ 1 khóa/bảng).	Không	Không
FOREIGN KEY	Tạo mối liên kết tới Primary Key của bảng khác.	Có	Có
UNIQUE	Đảm bảo mọi giá trị trong cột không bao giờ bị trùng lặp.	Có	Không
NOT NULL	Bắt buộc phải nhập dữ liệu khi thêm dòng mới.	Không	Có

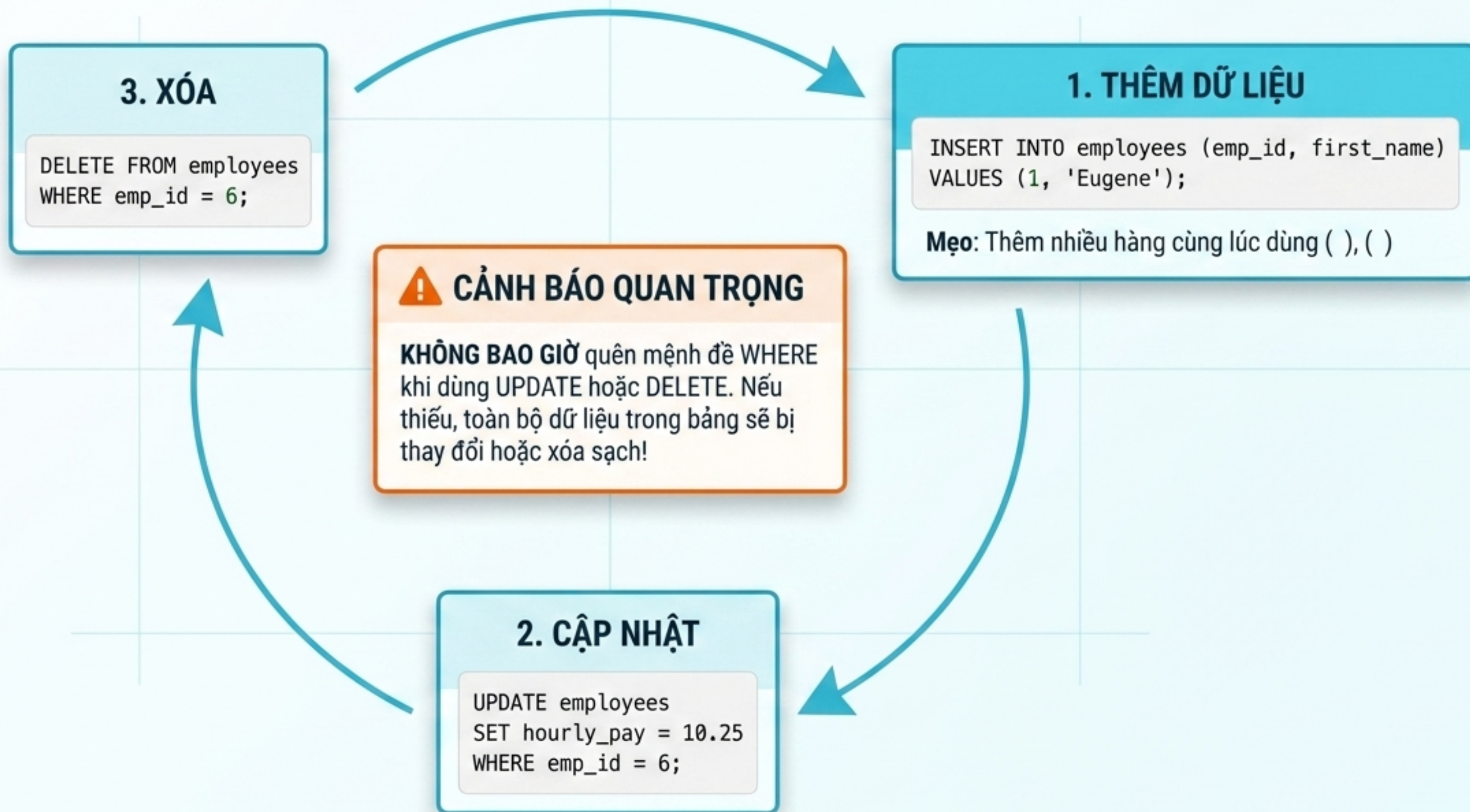
CHECK

Xác thực điều kiện logic
(VD: CHECK (hourly_pay >= 10))

DEFAULT

Gán giá trị dự phòng nếu không nhập
(VD: DEFAULT 0.00)

Thao Tác Dữ Liệu Cốt Lõi (DML - Vòng Đời Dữ Liệu)



Nghệ Thuật Lọc Dữ Liệu (DQL - Mệnh Đề WHERE)

Dữ Liệu Thô



Dữ Liệu Lọc

⚙️ Toán Tử Logic

AND (Đúng cả 2), OR (Đúng 1 trong 2), NOT (!=)

📁 Xử Lý Tập Hợp

IN ('cook', 'cashier') - Lọc giá trị trong danh sách

📅 Khoảng Giá Trị

BETWEEN '2023-01-01' AND '2023-01-31'

📄 Giá Trị Trống

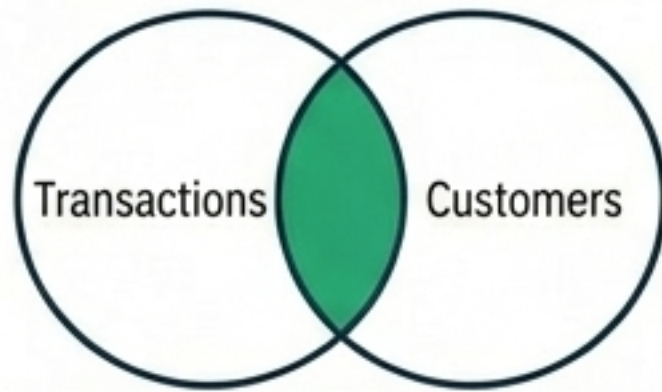
Dùng IS NULL hoặc IS NOT NULL (Tuyệt đối không dùng ~~= NULL~~)

🌈 Ký Tự Đại Diện (Wildcards) với LIKE

% : Đại diện cho 0, 1 hoặc nhiều ký tự (VD: LIKE 'S%' tìm tên bắt đầu bằng S)
_ : Đại diện cho đúng 1 ký tự (VD: LIKE '_a%' tìm tên có chữ 'a' ở vị trí số 2)

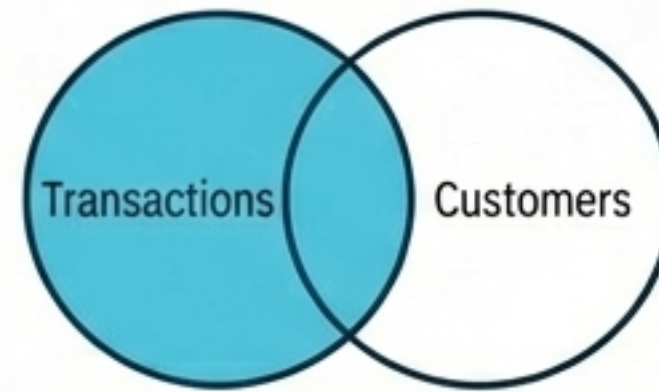
Liên Kết Các Bảng (Visualizing SQL Joins)

INNER JOIN



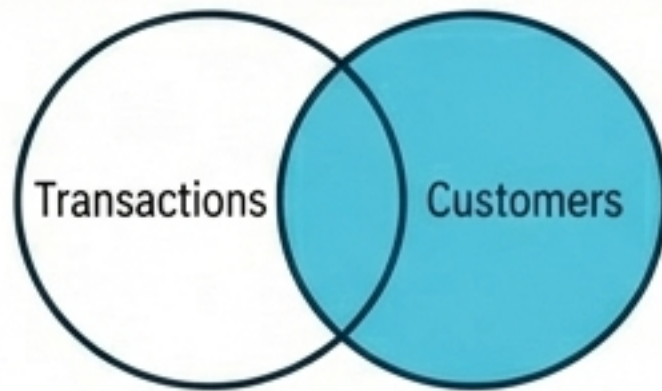
Chỉ lấy các dòng có **customer_id** khớp ở **CẢ HAI** bảng. (Khách hàng đã mua hàng).

LEFT JOIN



Lấy **TOÀN BỘ** dữ liệu bảng trái (Transactions), cộng thêm dữ liệu khớp ở bảng phải. Nếu không khớp, trả về **NULL** (VD: Khách mua vắng lại).

RIGHT JOIN



Lấy **TOÀN BỘ** khách hàng (Customers), kể cả người chưa từng mua hàng.

SELF JOIN



Join một bảng với chính nó. Dùng để biểu diễn hệ thống phân cấp (VD: Nhân viên A được quản lý bởi Nhân viên B).

```
FROM transactions AS a INNER JOIN customers AS b ON a.customer_id = b.customer_id
```

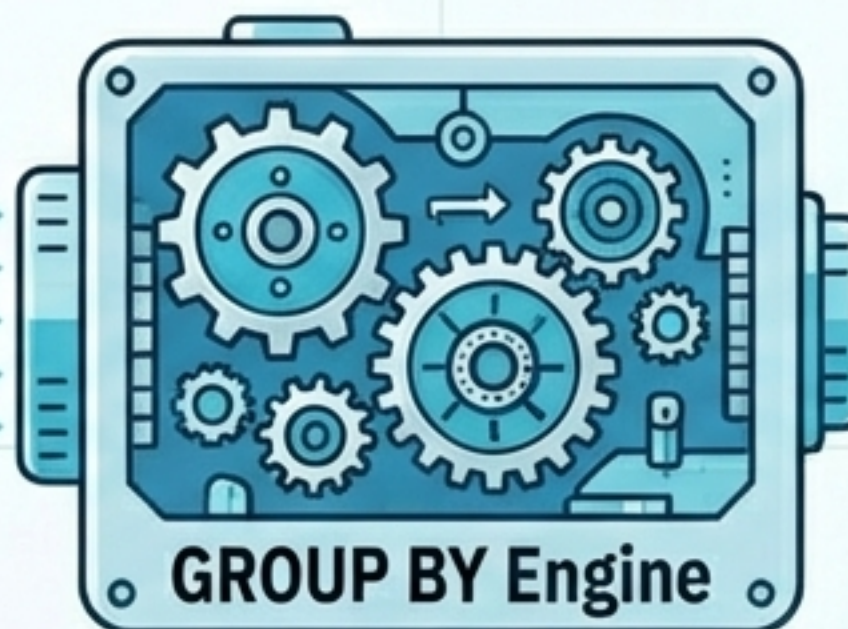

Tổng Hợp & Phân Tích Dữ Liệu (Aggregations)

 **COUNT():** Đếm dòng

 **SUM() / AVG():** Tổng / Trung bình

 **MAX() / MIN():** Lớn nhất / Nhỏ nhất

id	Region	customer_id	amount_id	Netral Sales
#9W	Meursti Neoab	Khách	B	\$1,200,500
#98315	Tanlp:	Apple	A	\$1,000
#238972	Product Cate.	Sales	B	\$450
#2355HX	Customer	Product Category	A	\$450
#5827i	Tissans	Apple	C	\$3,000
#96P	Bout Tnent	Customer	B	\$1,000



Sales by Region

Total Sales:	Avg Order Value:	Active Users:
\$1,200,500	\$450	15,000

Product Category Stats

Total Sales:	Avg Order Value:	Active Users:
\$1,200,500	\$450	15,000

Customer Segment

Active Users:	Avg Order Value:	Active Users:
15,000	\$450	15,000



WHERE

Lọc dữ liệu **TRƯỚC** khi nhóm.
(Chỉ áp dụng cho từng dòng đơn lẻ).

Golden Rule of Grouping

HAVING

Lọc dữ liệu **SAU** khi đã nhóm bằng **GROUP BY**
(VD: HAVING COUNT(amount) > 1).

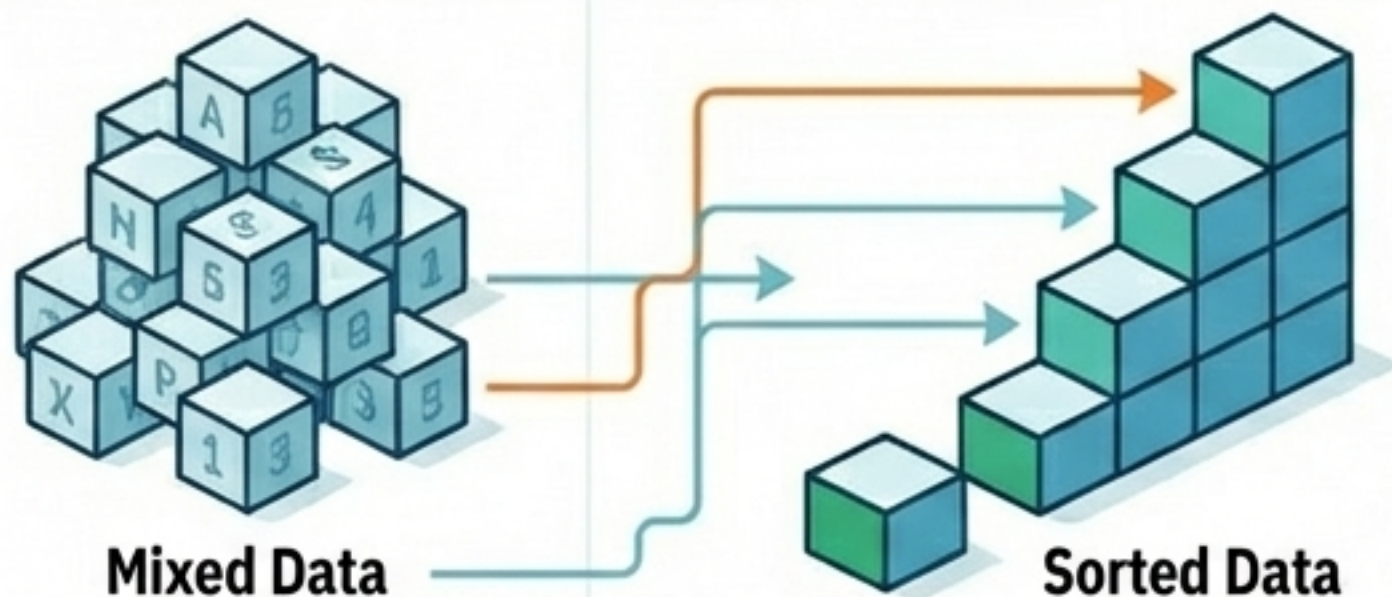


 **MẸO TỐI ƯU:**

Thêm WITH ROLLUP vào cuối mệnh đề GROUP BY để hệ thống tự động sinh ra một dòng tính Tổng Cộng (Grand Total) ở cuối bảng.

Sắp Xếp & Phân Trang (Sorting & Pagination)

↓ Sắp Xếp (ORDER BY)

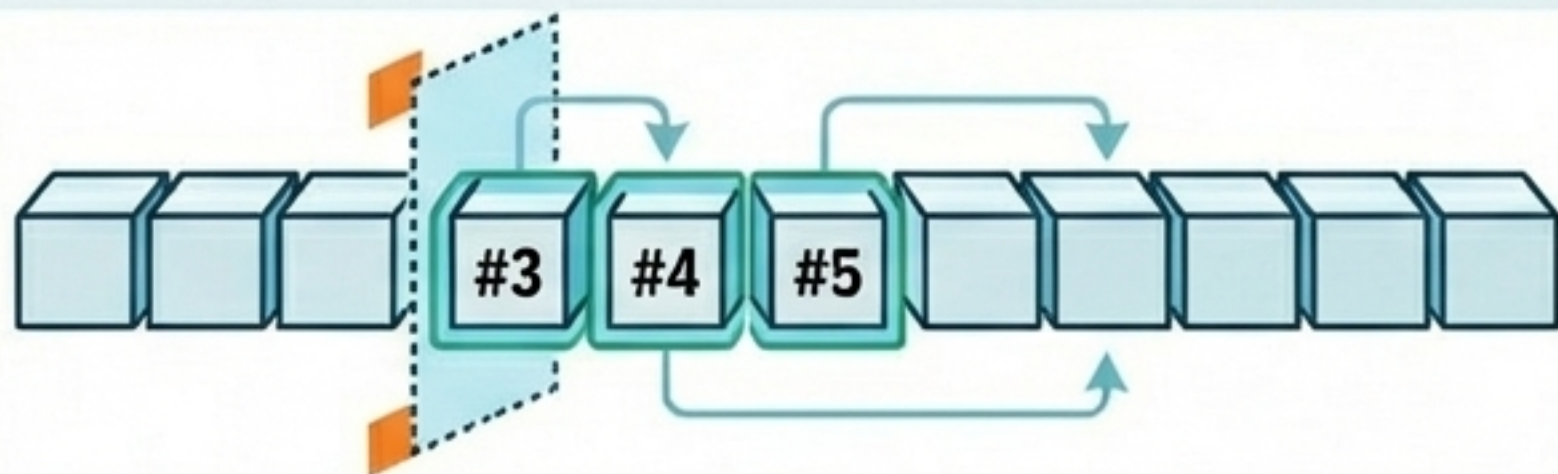


ORDER BY last_name **ASC**
(Tăng dần - Mặc định theo bảng chữ cái/số)

ORDER BY hourly_pay **DESC** (Giảm dần)

⚙️ **Nâng cao:** Sắp xếp theo nhiều cột. Nếu cột 1 trùng, xét tiếp cột 2 (ORDER BY amount DESC, customer_id ASC).

Phân Trang (LIMIT & OFFSET)



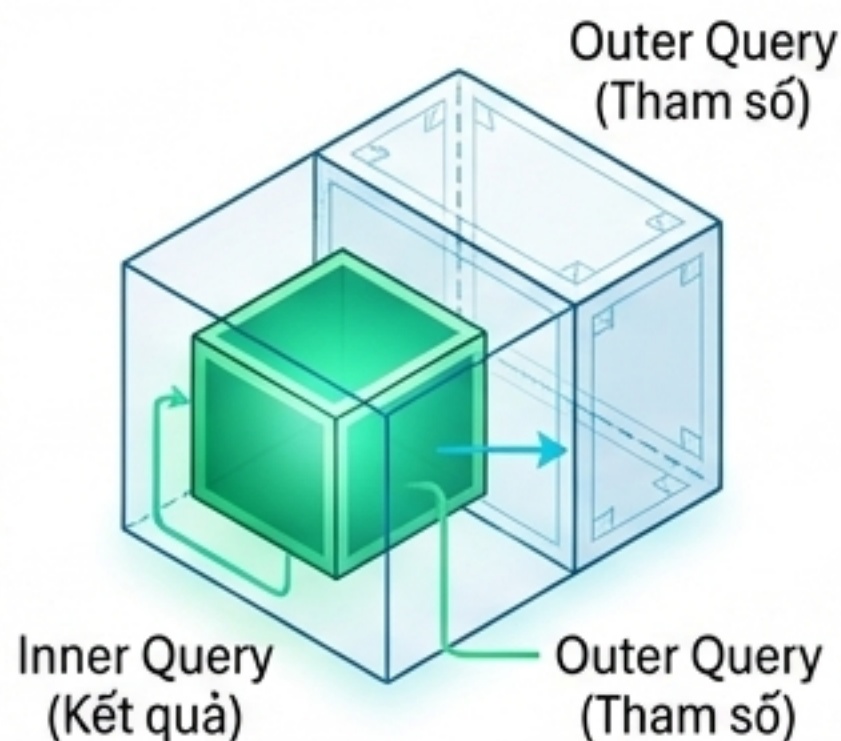
LIMIT x: Chỉ trả về tối đa x dòng (Hiển thị Top records).

LIMIT 10 OFFSET 20: Bỏ qua 20 dòng đầu tiên, lấy 10 dòng tiếp theo.

Subtext: Đây là công thức kinh điển để lập trình tính năng 'Next Page' trên Website.

Truy Vấn Phức Hợp (Subqueries vs. UNION)

Subquery (Truy vấn lồng nhau)



Lớp trong cùng (Inner Query) chạy trước, trả kết quả làm tham số cho lớp ngoài (Outer Query).

```
WHERE hourly_pay > (  
    SELECT AVG(hourly_pay) FROM employees  
)
```

Ghép Nối (UNION)



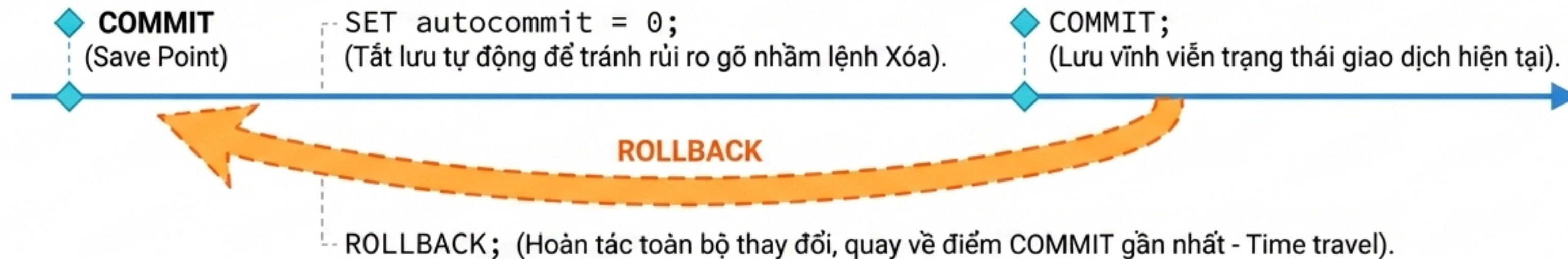
Ghép nối kết quả của 2 câu lệnh SELECT thành 1 bảng dọc duy nhất.

Quy tắc:

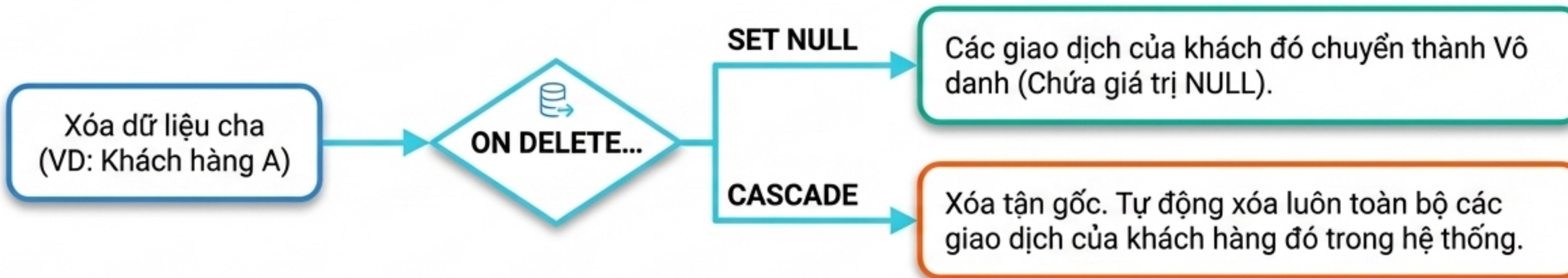
1. Hai câu lệnh bắt buộc phải trả về cùng số lượng cột.
2. Khác biệt: **UNION** (Tự động xóa các dòng trùng lặp) vs **UNION ALL** (Giữ lại toàn bộ dữ liệu trùng).

An Toàn & Toàn Vẹn Hệ Thống (Transactions & Integrity)

Quản Lý Giao Dịch (Transactions)



Hiệu Ứng Dây Chuyền (Foreign Key Deletion)



Tối Ưu Hóa Truy Vấn (Indexes & Views)

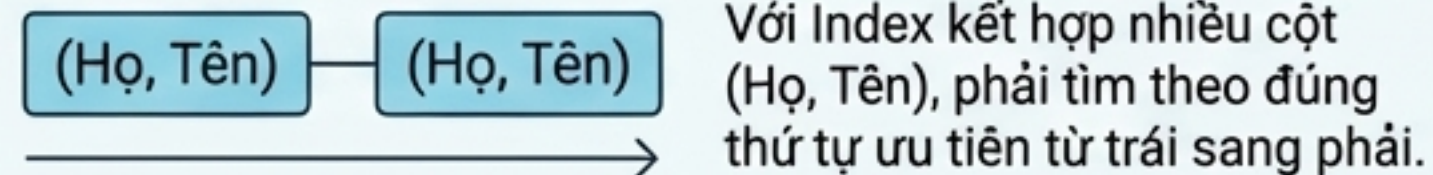
Chỉ Mục (Indexes - B-Tree Data Structure)



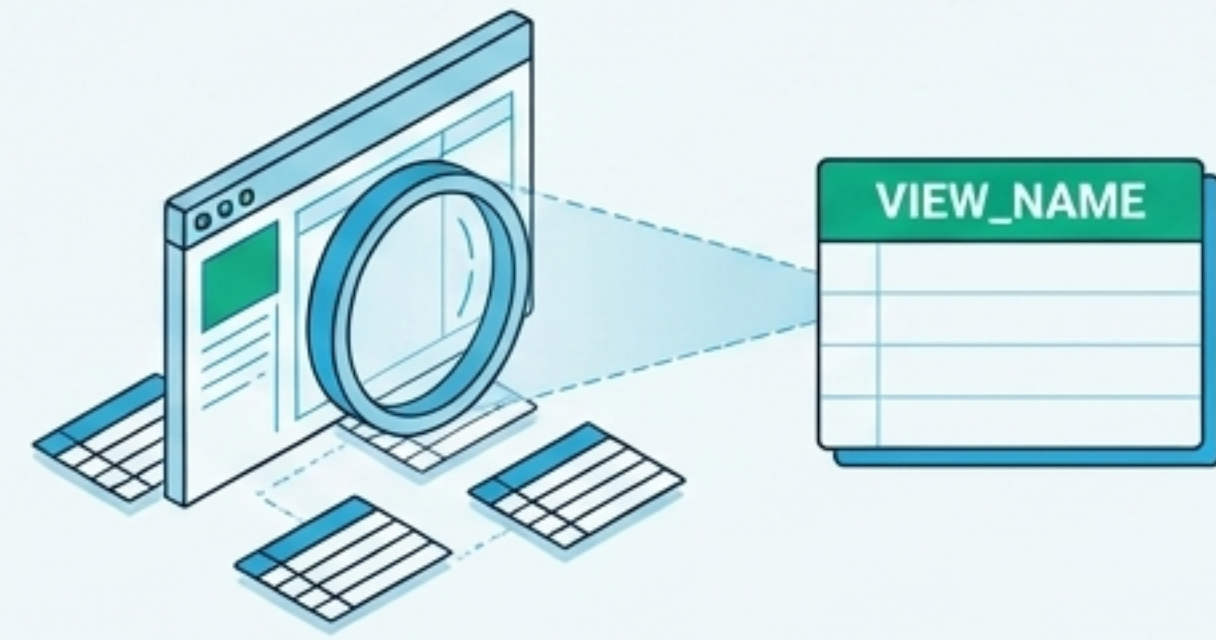
Cách hoạt động: Tạo mục lục cho cột để tăng tốc độ tìm kiếm thay vì phải quét toàn bộ bảng.

Trade-off: Tăng tốc SELECT / WHERE cực kỳ nhanh, nhưng làm chậm tốc độ ghi INSERT / UPDATE.

Quy tắc Leftmost Prefix:



Bảng Ảo (Views)



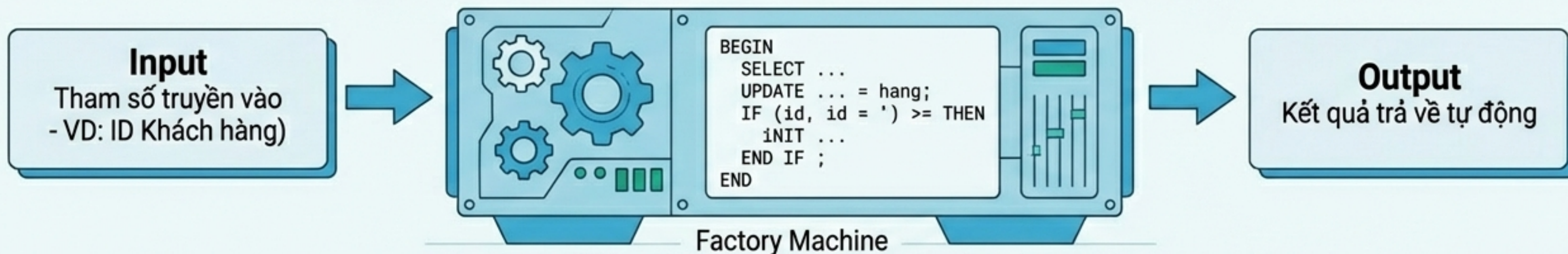
Khái niệm: Không phải bảng lưu dữ liệu vật lý. Là kết quả của một câu lệnh SELECT được lưu lại dưới dạng một bảng ảo.

Lợi ích: Ẩn đi các cột dữ liệu nhạy cảm, tái sử dụng logic phức tạp. Tự động cập nhật ngay khi dữ liệu ở bảng gốc thay đổi.

Lập Trình Cơ Sở Dữ Liệu (Stored Procedures)

Saved Logic Block

Một cỗ máy xử lý chứa hàng loạt mã SQL phức tạp



Khái Niệm & Cú Pháp

Lưu trữ các đoạn mã SQL phức tạp để tái sử dụng. Giảm tải băng thông mạng và tăng cường bảo mật.

DELIMITER \$\$: Thay đổi ký tự kết thúc lệnh tạm thời (từ ';' sang '\$\$') để MySQL không ngắt lệnh đột ngột giữa chừng.

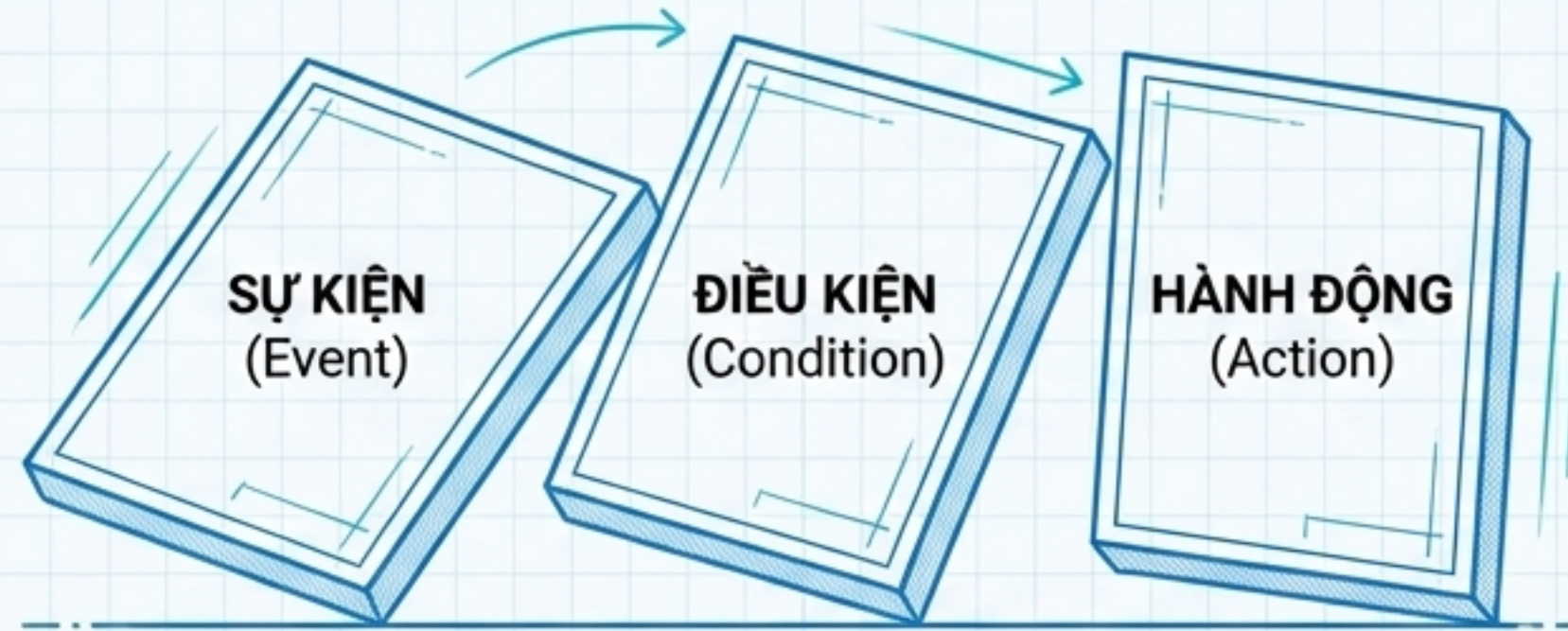
IN id INT : Định nghĩa tham số đầu vào cho Procedure.

Thực Thi (Execution)

CALL find_customer(1);

Hệ thống sẽ chạy toàn bộ logic ẩn bên trong để tìm và trả về thông tin chi tiết của khách hàng có ID = 1.

Tự Động Hóa Với Triggers (The Domino Effect)



Ma Trận Cấu Hình Trigger

-  **Thời điểm kích hoạt:** BEFORE (Trước khi) hoặc AFTER (Sau khi).
-  **Hành động lắng nghe:** INSERT, UPDATE, hoặc DELETE.
-  **Mục đích:** Chuyên dùng để kiểm tra tính hợp lệ của dữ liệu, xử lý lỗi tự động hoặc ghi nhật ký hệ thống (Audit logs).

Từ Khóa Ma Thuật: NEW vs OLD

- NEW: Đại diện cho dữ liệu vừa được nhập/sửa.
- OLD: Đại diện cho dữ liệu cũ chuẩn bị bị xóa/ghi đè.

```
SET NEW.salary = NEW.hourly_pay * 2080;
```

Tự động tính toán mức lương năm mới dựa trên mức lương theo giờ vừa được cập nhật.